

TD N° 2 : Architecture des ordinateurs

Exercice 1 :

Considérons le programme suivant :

```
list p=18f4550 , r=dec ;
include <p18f4550.inc>
org 0x0A
    loop    incf 70h
           goto loop
end
```

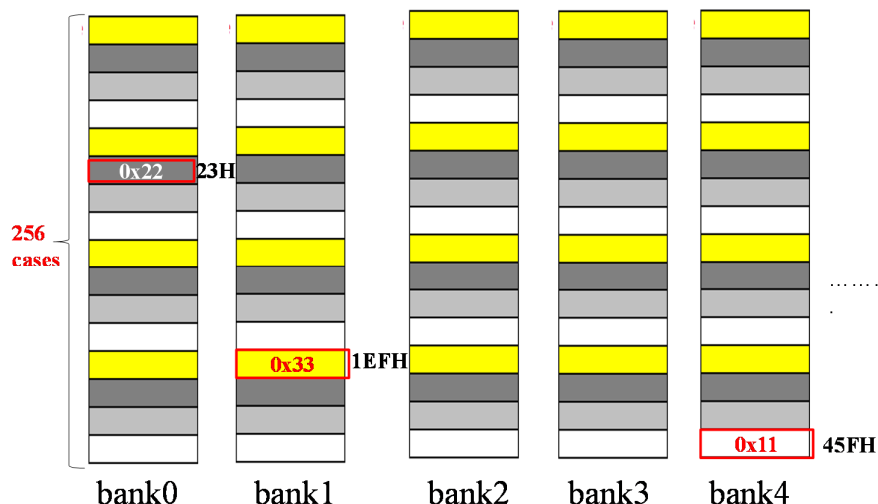
- 1) Peut-on changer la directive **org 0x0A** (ou org 0Ah) par **org 0xD5A** ? Expliquer.
- 2) Peut-on changer l'adresse de 70h (0x70) par **EEh** (0xEE) ? Expliquer.
- 3) Peut-on changer **0xEE** à **0x100** ? Expliquer et proposer une solution.
- 4) Changer l'adresse de **0x100** à **0xF92**. Commenter.

TABLE 5-1: SPECIAL FUNCTION REGISTER MAP : datasheet Page : 68

FB4h	CMCON	F94h	TRISC	F74h	UEP4
FB3h	TMR3H	F93h	TRISB	F73h	UEP3
FB2h	TMR3L	F92h	TRISA	F72h	UEP2
FB1h	T3CON	F91h	—(2)	F71h	UEP1
FB0h	SPBRGH	F90h	—(2)	F70h	UEP0

Exercice 2 : Adressage direct « Banked » & « Access Bank »

Soit les cases de la mémoire RAM d'un PIC18F4550 suivantes :



- 1) En utilisant la directive **equ**, déclarer trois variables nommées : variable1, variable2 et variable3 respectivement dans les cases mémoires d'adresses 23h, 1efh et 45fh.
- 2) En utilisant le registre spécial « **BSR** », mettez les valeurs indiquées dans la figure ci-dessus dans les cases correspondantes en faisant attention aux bancs auxquels elles appartiennent.
- 3) Permuter la valeur qui se trouve dans **variable1** avec celle qui se trouve dans **variable2** en utilisant la méthode d'adressage « *banked* ».
- 4) Même question en utilisant l'adressage direct « *access bank* ».
- 5) Compter le nombre de cycles nécessaire pour l'exécution du programme pour chaque type d'adressage.
- 6) Sachant que l'horloge externe utilisée avec ce PIC est de 4MHz, déterminer le temps d'exécution d'un seul cycle.
- 7) En déduire le temps d'exécution pour chaque programme.
- 8) Quel est l'avantage de la méthode d'adressage « *access bank* » par rapport à celle « *banked* » ?

Exercice 3 : Temporisation avec une boucle

Il arrive souvent qu'on désire introduire des temporisations pendant l'exécution d'un programme. Le PIC dispose de 3 **Timers** permettant de gérer le temps avec précision. Il y a une autre solution consistant à réaliser des temporisations à l'aide de **simples boucles**.

L'idée est : d'**initialiser une variable à une valeur donnée** et ensuite la décrémenter en **boucle** jusqu'à ce qu'elle **atteigne 0**. Connaissant le temps d'exécution de chaque instruction, on peut calculer le temps que mettra le processeur pour terminer la boucle de décrément.

- 1) Donner l'organigramme décrivant cette idée de temporisation.

On rappelle que pour une fréquence f_H de 4MHz un cycle processeur T_{cy} dure 4 cycles d'horloge T_H , soit 1 μ s. Les instructions durent presque toutes 1 cycle mais il y a certaines instructions qui durent 2 cycles comme: **call**, **return**, **goto** et **decfsz** si on saute l'instruction suivante.

Soit le code suivant : on met une valeur $N_1 = 250$ et on la décrémente jusqu'à 0.

movlw d'250'	→ 1 cycle
movwf N1 ; Initialiser N1	→ 1 cycle
boucle1	
nop ; Perdre 1 cycle	→ 1 cycle
decfsz N1, f ; Décrémenter N1	→ 1 ou 2 cycles
goto boucle1 ; Décrémenter N1	→ 2 cycles

- 2) Calculer la durée de ce programme T_1 avant de sortir de la **boucle 1**.
- 3) Calculer N_1 pour réaliser une temporisation $T_2 = 2\text{ms}$.
- 4) Cette solution est-elle réalisable ? Pourquoi ? Proposer des solutions.

Annexe

Les directives de l'assembleur sont des instructions qu'on ajoute dans le programme et qui seront interprétées par l'assembleur **MPASM**. Ce ne sont pas des instructions destinées au PIC.

1) La directive "list"

LIST : permet de définir un certain nombre de paramètres comme le processeur utilisé (**p**), la base par défaut pour les nombres (**r**) ainsi que d'autres paramètres.

Exemple: **LIST** *p=18F4550, r=dec (Hex, Oct, Bin)*

avec *r=dec*, les nombres sans spécification particulière seront considérés par l'assembleur comme des nombres décimaux.

2) la directive "include"

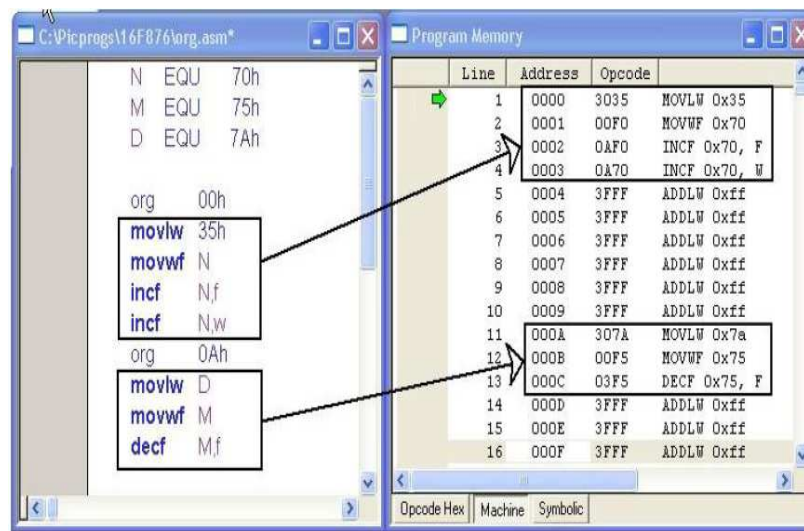
INCLUDE : permet d'insérer un fichier source. Par exemple le fichier *p18f4550.inc* contient la définition d'un certain nombre de constantes comme les noms des registres ainsi que les noms de certains bits.

Exemple: **INCLUDE** "p18f4550.inc "

3) Les directives "end" et "org":

END : indique la fin du programme.

ORG : (origine) définit la position dans la mémoire programme à partir de laquelle seront inscrites les instructions suivantes.



4) La directive "EQU" :

Permet de définir une constante ou une variable.

Exemple: **Var EQU 0x20**

Chaque fois que le compilateur rencontrera **Var**, il la remplacera par 0x20.

- Ça peut être une constante s'il s'agit d'une instruction avec adressage **Literal**: en effet, les instructions qui agissent sur une constante contiennent **L**, parce que chez Microchip, on appelle « **Literal** » ce genre d'adressage, chez d'autres constructeurs, on parle d'adressage immédiat.
- Ça peut être une adresse s'il s'agit d'une instruction avec adressage direct

Corrigé TD2

Corrigé Exercice 1 :

Correction question 3

```
list p=18f4550 , r=dec
include <p18f4550.inc>
org 0x0D5A
movlw 0x01 ; selectionner la banque 1
movwf BSR
loop incf 0x100,1,1
goto loop
End
```

Correction question 4

TRISA est un registre SFR, on ne peut pas l'utiliser pour effectuer des opérations intermédiaires. En effet, lorsqu'il est utilisé pour une application bien déterminée, il y a un risque de l'écraser. Par conséquent, il faut savoir manipuler les adresses de la RAM du PIC et déterminer les emplacements des registres SFR, puis utiliser les registres GPR.

Corrigé Exercice 2 :

- 1) list p=18f4550 , r=dec
include <p18f4550.inc>
;-----
variable1 EQU 0x23 ; adresse de la variable 1
variable2 EQU 0x1EF ; adresse de la variable 2
variable3 EQU 0x45F ; adresse de la variable 3
- 2) movlw H'22'
movwf variable1
movlb 0x01
movlw 0x33
movwf variable2
movlb 0x04
movlw H'11'
movwf variable3

1) list p=18F6550, r=dec
include <p18F6550.inc>
;-- declaration des variables--
variable1 equ 0x23
variable2 equ 0x1EF
variable3 equ 0x45F
Temp equ 0x77

2)

movlw 0x22	• Mettre la valeur 0x22 dans la case mémoire d'adresse "variable1" (dans bank 0)
movwf variable1	
movlw 0x01	• changer au bank 1 en mettant dans BSR "1".
movwf BSR	• on deux instructions peuvent être remplacées par movlb Nbank
movlw 0x33	• Mettre la valeur 0x33 dans la case mémoire d'adresse "variable2" (dans bank 1)
movwf variable2	
movlb 0x04	• Pointer sur le bank 4
movlw 0x11	• Mettre 0x11 dans variable 3.
movwf variable3	

3) Remplacement des contenus des "variable1" et "variable2" par la méthode "adressage banked"

movlb 0x00	• Passer au "bank 0"
movf variable1, 0	• Mettre le contenu de "variable1" dans "W"
movwf Temp	• Mettre le contenu de "W" dans la variable "Temp"
movlb 0x01	• Passer au "bank 1" où existe "variable2"
movf variable2, 0	• Mettre le contenu de "variable2" dans "W"
movlb 0	• Revenir au "bank 0" pour mettre le contenu de "W" (qui est le contenu de variable2) dans "variable1"
movwf variable1	
movf Temp, 0	• Mettre le contenu de "Temp" (qui est variable1) dans "W"
movlb 1	• Passer au "bank 1"
movwf variable2	• Pour mettre "W" dans "variable2"

END

4) Avec la méthode "Adressage Access bank"

movlb 1	• Tout d'abord, on se place dans "bank 1" où se trouve "variable2"
movf variable1, 0, 0	• Mettre le contenu de "variable1" dans "W" en utilisant l'adressage "Access bank : a=0"
movwf Temp, 0	• Mettre le contenu de "W" qui est "variable1" dans "Temp" (a=0)
movf variable2, 0	• Mettre "variable2" dans "W" (a=1: default, on est dans bank 1)
movwf variable1, 0	• Mettre le contenu de "W" dans "variable1" (a=0)
movf Temp, 0, 0	• Mettre dans "W" (d=0) le contenu de "Temp" (a=0)
movwf variable2	• Mettre dans "variable2" le contenu de "W". On est dans "bank 1" par défaut (a=1).

5) Avec adressage « banked »

```
list p=18f4550, r=dec
include <p18f4550.inc>
```

```
-----
;
```

```
variable1 EQU 0x23 ; adresse de la
variable1
variable2 EQU 0x1FE ; adresse de la
variable2
variable3 EQU 0x45F ; adresse de la
variable3
Temp EQU 0x5d;
```

```
-----
;
```

```
movlw H'22' ; 1 cycle
movwf variable1 ; 1 cycle
```

```
movlb 0x01 ; 1 cycle
movlw 0x33 ; 1 cycle
movwf variable2 ; 1 cycle
```

```
movlb 0x04 ; 1 cycle
movlw H'11' ; 1 cycle
movwf variable3 ; 1 cycle
```

```
-----
;
```

```
movlb 0x00 ; 1 cycle
movf variable1,0 ; 1 cycle
movwf Temp ; 1 cycle
movlb 0x01 ; 1 cycle
movf variable2,0 ; 1 cycle
movlb 0x00 ; 1 cycle
movwf variable1 ; 1 cycle
movf Temp,0 ; 1 cycle
movlb 0x01 ; 1 cycle
movwf variable2 ; 1 cycle
End
```

➡ 18 cycles avec Banked

6) Avec adressage "Access bank"

```
list p=18f4550, r=dec
include <p18f4550.inc>
```

```
-----
;
```

```
variable1 EQU 0x23 ; adresse de la
variable1
variable2 EQU 0x1FE ; adresse de la
variable2
variable3 EQU 0x45F ; adresse de la
variable3
Temp EQU 0x5d;
```

```
-----
;
```

➡ 15 cycles avec
Access Bank

```
movlw H'22' ; 1 cycle
movwf variable1 ; 1 cycle
```

```
movlb 0x01 ; 1 cycle
movlw 0x33 ; 1 cycle
movwf variable2 ; 1 cycle
```

```
movlb 0x04 ; 1 cycle
movlw H'11' ; 1 cycle
movwf variable3 ; 1 cycle
```

```
-----
;
```

```
movlb 0x01 ; 1 cycle
movf variable1,0,0 ; 1 cycle
movwf Temp,0 ; 1 cycle
movf variable2,0 ; 1 cycle
movwf variable1,0 ; 1 cycle
movf Temp,0,0 ; 1 cycle
movwf variable2 ; 1 cycle
```

```
End
```

Corrigé Exercice 3 :

Exercice 3 :

1°/ Organigramme :

```

graph TD
    Init[Initialiser N1] --> Dec[décrémenter N1]
    Dec --> Cond{N1=0?}
    Cond -- suite --> End[ ]
    Cond --> Dec
  
```

2°/ $T_1 = 1 + 1 + (1 + 1 + 2) \times (N_1 - 1) + (1 + 2)$

noy → 1
decfsz → 1
goto → 2

$T_1 = 4N_1 + 1$

noy → 1
decfsz → 2

$\Rightarrow T_1 = 1 + 1 + (1 + 1 + 2) \times 249 + (1 + 2)$

$T_1 = 4N_1 + 1$

$\Rightarrow T_1 = 1001 \text{ cycles}$

soit $T_1 = 1 \text{ ms}$

3°/ $T_1 = 4N_1 + 1 \Rightarrow N_1 = \frac{T_1 - 1}{4}$ avec $T_1 = 2 \text{ ms} = 2000 \mu\text{s} = 2000 \text{ cycles}$

$N_1 = \frac{2000 - 1}{4} \approx 500 \text{ cycles}$

4°/ solution non réalisable car $500 > 255$. solutions : on peut soit insérer d'autres instructions dans la boucle soit faire des boucles imbriquées.